

Contexto del problema y estado del arte

Pablo De la Fuente Arteaga, Rubén Torres Rodríguez y Alfonso González Avanzini

Lunes 4 de mayo de 2026

Contents

1	Contexto del problema	3
2	Estado del arte	3
3	Trabajos relacionados con el dataset BoT-IoT	4
3.1	Modelos clásicos de Machine Learning	4
3.2	Deep Learning aplicado a BoT-IoT	4
3.3	Selección de características	4
3.4	Autoencoders y aprendizaje de representaciones	5
3.5	Problema del desbalanceo de clases	5
3.6	Robustez y seguridad de los modelos	5
3.7	Resumen	5
4	Explicación del dataset	5
5	Preprocesamiento de los datos	7
5.1	Selección inicial de variables	8
5.2	Análisis de correlación y reducción de redundancia	8
5.3	Tratamiento de variables categóricas	8
5.4	Normalización de los datos	8
5.5	División del dataset	8
5.6	Tratamiento del desbalanceo de clases	8
5.7	Resumen del preprocesamiento	9
6	Diseño del modelo: Autoencoder + MLP	9
6.1	Autoencoder	9
6.2	Espacio latente	9
6.3	Clasificador MLP	10
6.4	Arquitectura del modelo	10
6.5	Ventajas del enfoque	10
6.6	Limitaciones	10
7	Bibliografía	11

1 Contexto del problema

En la era digital actual, la ciberseguridad se ha convertido en un pilar fundamental para la protección de datos e información sensible. Con el aumento exponencial de la conectividad a través de internet, dispositivos móviles y sistemas interconectados, las amenazas cibernéticas se han multiplicado y sofisticado considerablemente. Los ataques de piratas informáticos, malware, ransomware y phishing representan riesgos cada vez mayores para individuos, empresas e instituciones públicas. La exposición de información confidencial, robos de identidad y interrupciones en servicios críticos pueden ocasionar pérdidas económicas significativas y daños irreversibles a la reputación. Por esto, invertir en medidas de ciberseguridad robustas es esencial para mantener la integridad de los sistemas y proteger la privacidad en un mundo cada vez más digitalizado.

Además del impacto económico, la ciberseguridad es determinante para la estabilidad social e institucional. Los ataques a infraestructuras críticas como hospitales, sistemas de energía, entidades financieras y servicios públicos pueden comprometer la seguridad nacional y afectar directamente la vida cotidiana de millones de personas. Las organizaciones deben adoptar una cultura de seguridad digital, implementando protocolos de verificación, encriptación de datos, auditorías regulares y capacitación continua de sus empleados. Asimismo, los ciudadanos tienen la responsabilidad de adoptar buenas prácticas de higiene digital, como el uso de contraseñas seguras y la actualización periódica de sistemas. La ciberseguridad no es únicamente una cuestión tecnológica, sino una necesidad colectiva que requiere la colaboración de gobiernos, empresas y usuarios para construir un entorno digital más seguro y confiable.

2 Estado del arte

En los últimos tiempos, la técnica dominante para la detección de tráfico malicioso en redes la han constituido Sistemas de Detección de Intrusiones (IDS) basados en aprendizaje supervisado. Si bien estos métodos generan modelos con alta capacidad de generalización sobre las distribuciones de ataque, numerosos estudios modernos han mostrado también sus grandes limitaciones. La primera y más relevante, es su incapacidad para detectar ataques no observados durante el entrenamiento, el problema del zero-day. Esta carencia es especialmente grave en ciberseguridad, pues la distribución de los ataques es no estacionaria: los atacantes renuevan continuamente sus protocolos para evitar los detectores existentes. Especialmente, en un espacio IoT, con una inmensa cantidad de dispositivos y comunicaciones, resulta poco sustentable pensar que el conjunto de entrenamiento pueda capturar la plenitud de las amenazas. La segunda limitación es la dependencia de datos perfectamente etiquetados. El etiquetado del tráfico de red malicioso requiere un proceso lento y costoso de análisis forense, además propenso a errores. En escenarios reales, con un flujo de tráfico continuo, los modelos supervisados no pueden tratar nuevos datos no etiquetados y requieren de intervención humana. Este problema se ve agravado en el contexto IoT, donde modelos entrenados sobre un dataset difícilmente logran generalizar entornos distintos.

Estas limitaciones han incentivado a la comunidad investigadora a explorar alternativas que separen la fase de representación de la fase de clasificación. La finalidad no es sustituir el aprendizaje supervisado, sino reducir su exposición a las limitaciones estructurales, conservando su capacidad diferenciadora sobre las clases conocidas. En este contexto, aparece el autoencoder, que actúa como módulo de preprocesamiento y convierte el espacio de entrada de alta dimensión en una representación latente comprimida, así el clasificador supervisado trabajará con menor sensibilidad al ruido, menor coste de etiquetado y mayor eficacia ante variantes de patrones conocidos. Si bien esta arquitectura no resuelve el problema del zero-day, sí sobrelleva las limitaciones de dimensionalidad y coste que impiden el funcionamiento de los clasificadores supervisados puros en entornos IoT.

El dataset BOT-IoT (Koroniotis et al., 2019 Koroniotis et al. 2019) es uno de los más usados en estudios sobre la detección de intrusos en redes IoT. La diferencia principal con los datasets tradicionalmente usados es que fue creado en un entorno virtualizado que mezclaba tráfico de red legítimo de aparatos inteligentes con un flujo de ataques botnet modernos. Este conjunto de datos presenta un desbalanceo extremo entre tráfico de ataque y tráfico normal, observándose un ratio de 7687:1 (73,360,900 registros de ataque frente a 9543 registros de tráfico normal). Esta asimetría impone dos consecuencias metodológicas a abordar: evaluación

mediante F1-macro y AUC-ROC, y la incorporación de ponderación de clases en la función de pérdida del clasificador supervisado para evitar que el modelo colapse hacia la predicción de la clase mayoritaria.

La riqueza en clases del dataset (cinco categorías de tráfico y ocho subcategorías) refleja un cambio de enfoque de la detección binaria a la clasificación multiclase del tipo de ataque. Esto permite automatizar respuestas diferenciadas según el tipo de ataque detectado. Además, desde el punto de vista técnico, a diferencia de la clasificación binaria, donde el desbalance colapsa en una única clase mayoritaria de ataque, la clasificación multiclase obliga al modelo a discriminar entre categorías que comparten características estructurales de tráfico similares, como DDoS y DoS, mientras mantiene sensibilidad sobre clases extremadamente minoritarias como Theft.

La metodología más recurrente en los proyectos que usan este dataset consiste en emplear un autoencoder no para detectar anomalías, sino como módulo de reducción de dimensionalidad: se preentrena de manera no supervisada para extraer relaciones ocultas en el tráfico de red, y seguidamente, utilizando los resultados, se entrena un clasificador supervisado independiente. El estudio Wasswa et al. (arXiv, 2025) refleja mejor que ninguno este enfoque planteado. Compara explícitamente VAE-MLP con VAE-GCN y VAE-GAT. Sus resultados señalan que combinar el autoencoder variacional con un perceptrón multicapa supera significativamente el accuracy en clasificación multiclase que se obtiene cuando se combina con redes de grafos (99.72% frente a 86.42% y 89.46%).

En conjunto, la evidencia empírica disponible lleva hacia una metodología que no elimina las limitaciones del aprendizaje supervisado pero las ablanda en la medida de lo posible empleando la arquitectura de dos etapas descrita: un autoencoder como extractor de representaciones latentes, seguido de un clasificador supervisado que gestiona el desbalance mediante ponderaciones de clases en la función de pérdida.

3 Trabajos relacionados con el dataset BoT-IoT

El dataset BoT-IoT ha sido ampliamente utilizado en la literatura reciente como benchmark para la evaluación de sistemas de detección de intrusiones en entornos IoT. A continuación, se presentan algunos de los trabajos más relevantes que han empleado este conjunto de datos en tareas de clasificación.

3.1 Modelos clásicos de Machine Learning

Diversos trabajos han aplicado algoritmos tradicionales de Machine Learning sobre BoT-IoT. Por ejemplo, Abushwereb et al. Abushwereb et al. 2022 desarrollan un sistema de detección de intrusiones utilizando Apache Spark y algoritmos como Random Forest y Decision Trees, obteniendo altos valores de precisión.

De manera similar, Sibarani et al. Sibarani et al. 2023 utilizan varios algoritmos de Machine Learning para clasificar tráfico en el dataset BoT-IoT, demostrando que estos métodos siguen siendo competitivos cuando se combinan con un adecuado preprocesamiento.

3.2 Deep Learning aplicado a BoT-IoT

Con el auge del Deep Learning, numerosos trabajos han explorado arquitecturas más complejas. Alosaimi y Almutairi Alosaimi and Almutairi 2023 proponen un sistema de detección basado en técnicas de deep learning, logrando mejoras significativas en la detección de ataques en redes IoT.

Otros trabajos emplean modelos híbridos que combinan redes convolucionales (CNN) y redes recurrentes (LSTM), con el objetivo de capturar tanto patrones espaciales como temporales en el tráfico de red.

3.3 Selección de características

La alta dimensionalidad del dataset ha motivado el uso de técnicas de selección de características. Leevy et al. Leevy et al. 2021 proponen un enfoque basado en *ensemble feature selection* para detectar ataques de tipo *information theft*, demostrando mejoras en la precisión del modelo.

Asimismo, Özer et al. Ozer et al. 2021 proponen un enfoque basado en la selección de características para construir sistemas más eficientes, reduciendo el coste computacional sin sacrificar rendimiento.

3.4 Autoencoders y aprendizaje de representaciones

El uso de autoencoders ha sido explorado como una estrategia eficaz para reducir la dimensionalidad y aprender representaciones latentes de los datos. Estos modelos permiten capturar relaciones no lineales entre variables, lo que resulta especialmente útil en datasets complejos como BoT-IoT.

En general, los trabajos que emplean autoencoders siguen un enfoque en dos etapas: primero entrenan el autoencoder para reconstrucción, y posteriormente utilizan el espacio latente como entrada para un clasificador.

3.5 Problema del desbalanceo de clases

Uno de los problemas más relevantes del dataset es el desbalanceo entre clases. Atuhurra et al. Atuhurra et al. 2024 analizan este problema y proponen el uso de técnicas como SMOTE para mejorar la detección de clases minoritarias.

3.6 Robustez y seguridad de los modelos

Más allá de la precisión, algunos trabajos han estudiado la robustez de los modelos frente a ataques adversarios. Papadopoulos et al. Papadopoulos et al. 2021 analizan cómo modelos entrenados con BoT-IoT pueden ser vulnerables a ataques como *label poisoning* o *fast gradient sign method*.

3.7 Resumen

En conjunto, la literatura muestra una evolución desde métodos clásicos hacia modelos basados en Deep Learning y aprendizaje de representaciones. Sin embargo, persisten desafíos importantes como el desbalanceo de clases, la redundancia de variables y la robustez frente a ataques adversarios.

El presente trabajo se enmarca dentro de esta línea, proponiendo el uso de autoencoders para la extracción de características y un clasificador MLP en el espacio latente.

4 Explicación del dataset

Contamos con las siguientes variables:

- **pkSeqID**: Identificador de fila.
- **sbytes**: Recuento de bytes desde el origen hasta el destino.
- **dbytes**: Recuento de bytes de destino a origen.
- **Stime**: Hora de inicio de la grabación.
- **flgs**: Indicadores del estado de flujo observados en las transacciones.
- **flgs_number**: Representación numérica de los indicadores de características.
- **rate**: Total de paquetes por segundo en la transacción.
- **srates**: Paquetes de origen a destino por segundo.
- **drates**: Paquetes de destino a origen por segundo.
- **Proto**: Representación textual de los protocolos de transacción presentes en el flujo de red.
- **proto_number**: Representación numérica de la característica proto.

- **TnBPSrcIP**: Número total de bytes por dirección IP de origen.
- **TnBPDstIP**: Número total de bytes por dirección IP de destino.
- **TnP_PSrcIP**: Número total de paquetes por dirección IP de origen.
- **TnP_PDstIP**: Número total de paquetes por dirección IP de destino.
- **saddr**: Dirección IP de origen.
- **sport**: Número de puerto de origen.
- **dport**: Número de puerto de destino.
- **daddr**: Dirección IP de destino.
- **TnP_PerProto**: Número total de paquetes por protocolo.
- **TnP_PerDport**: Número total de paquetes por puerto de destino.
- **pkts**: Número total de paquetes en la transacción.
- **AR_P_Proto_P_SrcIP**: Tasa media por protocolo por dirección IP de origen (paquetes/duración).
- **AR_P_Proto_P_DstIP**: Tasa media por protocolo por dirección IP de destino.
- **N_IN_Conn_P_SrcIP**: Número de conexiones entrantes por dirección IP de origen.
- **N_IN_Conn_P_DstIP**: Número de conexiones entrantes por dirección IP de destino.
- **bytes**: Número total de bytes en la transacción.
- **state**: Estado de la transacción.
- **state_number**: Representación numérica del estado de una característica.
- **ltime**: Último resultado.
- **seq**: Número de secuencia de Argus.
- **dur**: Duración total del registro.
- **mean**: Duración media de los registros agregados.
- **stddev**: Desviación estándar de los registros agregados.
- **sum**: Duración total de los registros agregados.
- **min**: Duración mínima de los registros agregados.
- **max**: Duración máxima de los registros agregados.
- **AR_P_Proto_P_Sport**: Tarifa media por protocolo por puerto de origen.
- **AR_P_Proto_P_Dport**: Tasa media por protocolo por puerto de destino.
- **Pkts_P_State_P_Protocol_P_DstIP**: Paquetes agrupados por estado y protocolo según la IP de destino.
- **Pkts_P_State_P_Protocol_P_SrcIP**: Paquetes agrupados por estado y protocolo según la IP de origen.
- **attack**: Etiqueta de clase (0 = Normal, 1 = Ataque).
- **category**: Categoría de tráfico (**variable objetivo**).

- **subcategory**: Subcategoría de tráfico.
- **spkts**: Recuento de paquetes de origen a destino.
- **dpkts**: Recuento de paquetes de destino a origen.

Nuestra variable objetivo es **category** encargada de clasificar el tipo de ataque. Cuenta con las siguientes 5 categorías:

- **DoS** → ataque de denegación de servicio: el objetivo del cibercriminal es dejar inutilizado el servidor por un periodo largo de tiempo. Para ello el atacante realiza miles de peticiones por segundo al servidor haciendo que colapse.

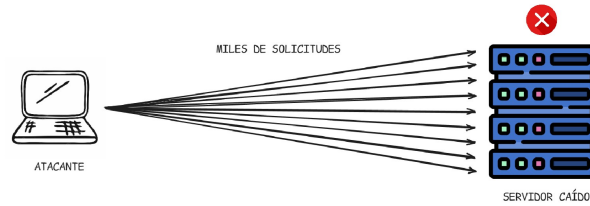


Figure 1: Funcionamiento de un ataque DoS.

- **DDoS** → misma técnica que la anterior con la peculiaridad de que se realiza desde muchas IPs distintas, haciendo uso de bots, y con ello constituyendo una **botnet**.

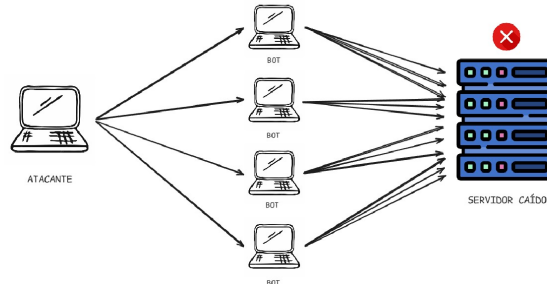


Figure 2: Funcionamiento de un ataque DDoS.

- **Normal** → clasificado como no ataque.
- **Reconnaissance** → los ataques de reconocimiento (también llamados reconnaissance o fase de footprinting) son la etapa inicial en la que un atacante recopila información sobre un objetivo antes de intentar comprometerlo. En esta fase no necesariamente se produce una intrusión directa, sino que se busca identificar datos como direcciones IP, dominios, servicios expuestos, sistemas operativos, etc.
- **Theft** → robo o exfiltración de información sensible (datos personales, credenciales, propiedad intelectual, etc).

5 Preprocesamiento de los datos

El preprocesamiento de los datos constituye una etapa fundamental en el desarrollo de modelos de aprendizaje automático, especialmente en datasets complejos y de alta dimensionalidad como el utilizado en este trabajo. En este caso, el dataset presenta características propias de tráfico de red en entornos IoT, incluyendo redundancia entre variables, presencia de valores categóricos y un notable desbalanceo entre clases.

A continuación, se detallan las distintas etapas de preprocesamiento aplicadas.

5.1 Selección inicial de variables

En una primera fase, se llevó a cabo una revisión de las variables disponibles con el objetivo de eliminar aquellas que no aportan información relevante para el problema de clasificación. En particular, se eliminaron variables identificadoras o con alta dependencia del contexto específico, tales como direcciones IP, identificadores de flujo o marcas temporales exactas.

Este tipo de variables puede introducir ruido en el modelo y favorecer el sobreajuste, ya que no generalizan adecuadamente a nuevos datos.

5.2 Análisis de correlación y reducción de redundancia

A partir del análisis de la matriz de correlación, se identificaron grupos de variables altamente correlacionadas, especialmente aquellas relacionadas con volumen de tráfico y estadísticas agregadas.

La presencia de estas correlaciones elevadas indica redundancia en el dataset, lo que puede afectar negativamente al rendimiento del modelo al aumentar la dimensionalidad sin aportar información adicional.

Para mitigar este problema, se consideraron dos enfoques:

- Eliminación de variables altamente correlacionadas, manteniendo únicamente una de cada grupo.
- Uso de técnicas de reducción de dimensionalidad, como autoencoders, capaces de aprender representaciones compactas de los datos.

En este trabajo se opta principalmente por el segundo enfoque, permitiendo que el modelo aprenda automáticamente las relaciones entre variables.

5.3 Tratamiento de variables categóricas

El dataset contiene variables categóricas, como protocolos o estados de conexión, que no pueden ser utilizadas directamente por modelos basados en redes neuronales.

Para su transformación, se aplicó la técnica de *One-Hot Encoding* (OHE), que convierte cada categoría en una variable binaria. Este procedimiento permite representar la información categórica en formato numérico sin introducir relaciones artificiales entre categorías.

No obstante, esta transformación incrementa la dimensionalidad del dataset, lo que refuerza la necesidad de aplicar técnicas de reducción posteriores.

5.4 Normalización de los datos

Dado que las variables presentan diferentes escalas (por ejemplo, número de paquetes frente a tasas de transferencia), se aplicó una normalización utilizando **StandardScaler**, transformando los datos para que tengan media cero y varianza unitaria.

Este paso es especialmente importante en modelos basados en redes neuronales, ya que mejora la estabilidad del entrenamiento y favorece la convergencia del modelo.

5.5 División del dataset

El conjunto de datos se dividió en conjuntos de entrenamiento y prueba utilizando una proporción 80/20. Además, se aplicó una estrategia de estratificación para preservar la distribución de clases en ambos subconjuntos.

Esta división permite evaluar el rendimiento del modelo en datos no vistos, garantizando una estimación más realista de su capacidad de generalización.

5.6 Tratamiento del desbalanceo de clases

El dataset presenta un desbalanceo significativo entre clases, con algunas categorías altamente representadas y otras con un número reducido de muestras.

Para abordar este problema, se utilizaron pesos de clase durante el entrenamiento del modelo, penalizando en mayor medida los errores cometidos en clases minoritarias. Este enfoque permite mejorar el rendimiento del modelo en dichas clases sin necesidad de modificar directamente la distribución de los datos.

5.7 Resumen del preprocesamiento

En conjunto, el preprocesamiento aplicado permite transformar el dataset original en una representación adecuada para el entrenamiento de modelos de Deep Learning. Se han abordado problemas clave como la redundancia de variables, la presencia de datos categóricos, la normalización de escalas y el desbalanceo de clases.

Estas transformaciones resultan esenciales para garantizar un entrenamiento estable y un buen rendimiento del modelo propuesto.

6 Diseño del modelo: Autoencoder + MLP

Con el objetivo de abordar la alta dimensionalidad y redundancia presente en el dataset, se propone un modelo híbrido basado en un autoencoder para la extracción de características, seguido de un perceptrón multicapa (MLP) para la clasificación.

6.1 Autoencoder

Un autoencoder es una red neuronal no supervisada cuyo objetivo es aprender una representación comprimida de los datos de entrada. Está compuesto por dos partes principales: un codificador (*encoder*) y un decodificador (*decoder*).

El encoder transforma la entrada $x \in \mathbb{R}^n$ en una representación latente $z \in \mathbb{R}^d$, donde $d < n$:

$$z = f_{\theta}(x)$$

donde f_{θ} representa la función paramétrica del encoder.

El decoder intenta reconstruir la entrada original a partir del espacio latente:

$$\hat{x} = g_{\phi}(z)$$

donde g_{ϕ} es la función del decoder.

El modelo se entrena minimizando el error de reconstrucción, típicamente mediante el error cuadrático medio:

$$\mathcal{L}_{rec} = \|x - \hat{x}\|^2$$

El objetivo de este proceso es que el espacio latente capture las características más relevantes de los datos, eliminando redundancias y ruido.

6.2 Espacio latente

El espacio latente constituye una representación de menor dimensión que preserva la información esencial del dataset. En este trabajo, se selecciona una dimensión reducida para forzar al modelo a aprender estructuras significativas en los datos.

Esta representación es especialmente útil en datasets como BoT-IoT, donde muchas variables están altamente correlacionadas.

6.3 Clasificador MLP

Una vez obtenido el espacio latente, se utiliza como entrada a un perceptrón multicapa (MLP) para realizar la clasificación multiclase.

El MLP implementa una función:

$$\hat{y} = h_{\psi}(z)$$

donde h_{ψ} es una red neuronal con varias capas densas y activaciones no lineales.

La salida del modelo se obtiene mediante una función *softmax*, que permite interpretar las predicciones como probabilidades sobre las distintas clases:

$$P(y = k \mid z) = \frac{e^{a_k}}{\sum_j e^{a_j}}$$

El modelo se entrena utilizando la función de pérdida de entropía cruzada categórica:

$$\mathcal{L}_{cls} = - \sum_k y_k \log(\hat{y}_k)$$

6.4 Arquitectura del modelo

El encoder está compuesto por varias capas densas que reducen progresivamente la dimensionalidad de los datos, mientras que el decoder sigue una estructura simétrica para reconstruir la entrada.

El clasificador MLP consiste en varias capas densas con activación ReLU, finalizando en una capa softmax con tantas neuronas como clases.

6.5 Ventajas del enfoque

El uso de un autoencoder presenta varias ventajas:

- Reducción de dimensionalidad sin necesidad de técnicas lineales como PCA.
- Eliminación de ruido y redundancias en los datos.
- Mejora del rendimiento del clasificador al trabajar con representaciones más compactas.
- Capacidad de capturar relaciones no lineales entre variables.

Además, la separación entre extracción de características y clasificación permite una mayor flexibilidad en el diseño del sistema.

6.6 Limitaciones

A pesar de sus ventajas, este enfoque también presenta limitaciones:

- El autoencoder puede aprender representaciones no óptimas para la clasificación si se entrena únicamente con reconstrucción.
- Existe riesgo de sobreajuste si el modelo es demasiado complejo.
- La elección de la dimensión del espacio latente es crítica.

7 Bibliografia

References

- Abushwereb, Mohammad et al. (2022). “Intrusion Detection System for IoT Using Apache Spark and Machine Learning”. In: *arXiv preprint arXiv:2203.04347*.
- Alosaimi, Waleed and Khalid Almutairi (2023). “Deep Learning-Based Intrusion Detection System for IoT Networks”. In: *Sensors* 23.21.
- Atuhurra, John et al. (2024). “Handling class imbalance in IoT intrusion detection datasets”. In: *arXiv preprint arXiv:2403.18989*.
- Koroniotis, Nickolaos et al. (2019). “Towards the development of realistic botnet dataset in the internet of things for network forensic analytics”. In: *Future Generation Computer Systems* 100, pp. 779–796.
- Leevy, Joffrey et al. (2021). “IoT information theft detection using ensemble feature selection”. In: *Journal of Big Data* 8.1.
- Ozer, Esra et al. (2021). “Feature selection for intrusion detection systems in IoT using BoT-IoT dataset”. In: *International Journal of Distributed Sensor Networks*.
- Papadopoulos, Alexandros et al. (2021). “Adversarial attacks against machine learning models for intrusion detection”. In: *arXiv preprint arXiv:2104.12426*.
- Sibarani, Evan et al. (2023). “Machine Learning-based Intrusion Detection on BoT-IoT Dataset”. In: *Journal of Applied Intelligent System*.